



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش پیشرفت پروژه اینترنت اشیا - هفته دوم

IOT Project Progress Report - Week 2

اعضای گروه

علی فتحی

آرمین فارسی

امیررضا بهرامی

نام درس

اینترنت اشیا

تابستان ۱۴۰۵

نام استاد درس

دکتر اجلالی

طراحی و پیاده‌سازی سامانه مدیریت OTA برای ESP32

در هفته دوم، تمرکز پروژه از آماده‌سازی ارتباط اولیه به اجرای واقعی فرآیند OTA منتقل شد. ابتدا ظرفیت حافظه Flash مورد و ساختار پارتیشن‌ها بررسی شد، سپس جدول پارتیشن مناسب برای نگهداری چند نسخه ثابت‌افزار طراحی و روی برد اعمال شد. در ادامه، پاسخ سرور برای وجود نسخه جدید در سمت برد پردازش شد، فایل ثابت‌افزار از سرور دریافت و روی پارتیشن مقصد نوشته شد و پس از تغییر پارتیشن بوت، برد با نسخه جدید راه‌اندازی شد. در پایان نیز وضعیت مراحل به‌روزرسانی در سرور ثبت شد و اجرای موفق نسخه جدید پس از راه‌اندازی مجدد تأیید گردید.

۱. بررسی حافظه Flash و طراحی ساختار پارتیشن‌های OTA

در شروع هفته دوم، پیش از پیاده‌سازی دریافت و نصب ثابت‌افزار، اندازه واقعی حافظه Flash مورد بررسی شد. نتیجه بررسی نشان داد حافظه فیزیکی مورد برابر 4MB است، در حالی که تنظیم اولیه پروژه روی 2MB قرار داشت. همچنین اندازه ثابت‌افزار در این مرحله حدود 885792 بایت بود. بنابراین تنظیم ظرفیت حافظه در پروژه اصلاح شد و یک جدول پارتیشن اختصاصی برای اجرای OTA طراحی گردید.

```
Flash Memory Information:
=====
Manufacturer: 68
Device: 4016
Detected flash size: 4MB
Flash voltage set by a strapping pin: 3.3V
Hard resetting via RTS pin...
```

شکل ۱: خروجی بررسی حافظه Flash مورد و تشخیص ظرفیت 4MB

۱-۱. طراحی جدول پارتیشن

ساختار قبلی تنها شامل یک پارتیشن برنامه با عنوان factory بود و امکان نگهداری هم‌زمان نسخه فعلی و نسخه جدید را فراهم نمی‌کرد. جدول جدید شامل پارتیشن‌های factory، ota_0، ota_1 و otadata است. فایل partitions.csv به صورت زیر تعریف شد:

#	Name,	Type,	SubType,	Offset,	Size,	Flags
1	nvs,	data,	nvs,	0x9000,	0x4000,	
2	otadata,	data,	ota,	0xd000,	0x2000,	
3	phy_init,	data,	phy,	0xf000,	0x1000,	
4	factory,	app,	factory,	0x10000,	0x140000,	
5	ota_0,	app,	ota_0,	0x150000,	0x150000,	
6	ota_1,	app,	ota_1,	0x2A0000,	0x150000,	

در این ساختار، نسخه اولیه می‌تواند از پارتیشن factory اجرا شود و نسخه‌های جدید به صورت متناوب در ota_0 و ota_1 قرار گیرند. پارتیشن otadata نیز اطلاعات لازم برای انتخاب پارتیشن بوت بعدی را نگهداری می‌کند.

```
Parsing binary partition input...
Verifying table...
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
nvs,data,nvs,0x9000,16K,
otadata,data,ota,0xd000,8K,
phy_init,data,phy,0xf000,4K,
factory,app,factory,0x10000,1280K,
ota_0,app,ota_0,0x150000,1344K,
ota_1,app,ota_1,0x2a0000,1344K,
```

شکل ۲: جدول پارتیشن جدید شامل `ota_1`، `ota_0`، `otadata`، `factory`

۲-۱. بررسی پارتیشن در حال اجرا و مقصد به روزرسانی

برای اطمینان از عملکرد صحیح ساختار جدید، در سمت بورد پارتیشن در حال اجرا و پارتیشن انتخاب شده برای به روزرسانی بعدی خوانده می شوند. بخش اصلی این بررسی به صورت زیر است:

```
1 const esp_partition_t *running =
2     esp_ota_get_running_partition();
3
4 const esp_partition_t *next =
5     esp_ota_get_next_update_partition(NULL);
6
7 ESP_LOGI(TAG, "Running partition: %s", running->label);
8 ESP_LOGI(TAG, "Next OTA partition: %s", next->label);
```

در اجرای اولیه، برنامه از `factory` اجرا شد و `ota_0` به عنوان مقصد اولین به روزرسانی انتخاب شد. پس از نصب نسخه جدید روی `ota_0`، پارتیشن بعدی به صورت خودکار `ota_1` تعیین شد.

```
I (583) ota_manager: Running partition: factory
I (583) ota_manager: Running partition address: 0x10000
I (593) ota_manager: Running partition size: 1310720 bytes
I (603) ota_manager: Next OTA partition: ota_0
I (603) ota_manager: Next OTA partition address: 0x150000
I (603) ota_manager: Next OTA partition size: 1376256 bytes
```

شکل ۳: خروجی Serial Monitor و نمایش پارتیشن در حال اجرا و پارتیشن مقصد OTA

۲. تکمیل بررسی نسخه جدید و پردازش پاسخ سرور

در هفته اول، بورد تنها درخواست بررسی نسخه را برای سرور ارسال می کرد. در هفته دوم ساختار پاسخ سرور کامل تر پردازش شد تا اطلاعات مورد نیاز برای نصب نسخه جدید در اختیار بورد قرار گیرد. برای پردازش پاسخ JSON از کتابخانه `cJSON` استفاده شد.

اطلاعات مربوط به هر عملیات به‌روزرسانی در ساختار زیر نگهداری می‌شود:

```
1 typedef struct
2 {
3     bool update_available;
4
5     int update_id;
6     int firmware_id;
7
8     char target_version[OTA_VERSION_SIZE];
9     char file_url[OTA_URL_SIZE];
10    char sha256[OTA_SHA256_SIZE];
11
12    size_t file_size;
13 } ota_update_info_t;
```

در صورت وجود نسخه جدید، سرور شناسه عملیات، نسخه مقصد، آدرس دریافت فایل، اندازه فایل و مقدار SHA-256 را در پاسخ قرار می‌دهد. بخش مربوط به استخراج اطلاعات اصلی از پاسخ به صورت زیر پیاده‌سازی شد:

```
1 update_info->update_id = update_id->valueint;
2 update_info->firmware_id = firmware_id->valueint;
3 update_info->file_size = (size_t)file_size->valuedouble;
4
5 snprintf(
6     update_info->target_version,
7     sizeof(update_info->target_version),
8     "%S",
9     target_version->valuestring
10 );
11
12 snprintf(
13     update_info->file_url,
14     sizeof(update_info->file_url),
15     "%S",
16     file_url->valuestring
17 );
```

به این ترتیب مورد پس از دریافت پاسخ، می‌تواند مستقیماً آدرس فایل ثابت‌افزار مقصد را برای مرحله دریافت و نصب استفاده کند.

```
I (9453) main: Update available!
I (9453) main: Target version: 1.1.0
I (9453) main: Firmware ID: 1
I (9463) main: Firmware size: 892336 bytes
I (9463) main: Firmware URL: http://192.168.45.216:8000/api/firmwares/1/download
I (9473) main: SHA-256: 7d354ed0b2cc6df060ed9af779c3ee9428a33789d611d292b5aa749d8efa6fb7
```

شکل ۴: دریافت پاسخ سرور در مورد و نمایش نسخه مقصد، اندازه فایل و آدرس دریافت ثابت‌افزار

۳. آماده‌سازی نسخه‌های جدید ثابت‌افزار در سرور و پنل

برای آزمایش واقعی فرآیند OTA، نسخه‌های جدید ثابت‌افزار بدون انتقال مستقیم از طریق USB ساخته شدند. فایل‌های نسخه‌های 1.1.0 و 1.2.0 در پوشه releases نگهداری شده و از طریق پنل مدیریتی روی سرور بارگذاری شدند. در سمت سرور، هنگام بارگذاری فایل، نوع فایل بررسی شده و پس از ذخیره آن، اندازه و مقدار SHA-256 محاسبه و در پایگاه داده ثبت می‌شود. بخش اصلی این عملیات به صورت زیر است:

```
1 with open(target, "wb") as out:
2     shutil.copyfileobj(file.file, out)
3
4 size = target.stat().st_size
5
6 cur = conn.execute(
7     """INSERT INTO firmwares
8         (version, file_name, file_path, file_size, sha256, changelog)
9         VALUES (?, ?, ?, ?, ?, ?)""",
10    (
11        version,
12        file.filename or target.name,
13        str(target),
14        size,
15        sha256_of(target),
16        changelog.strip() or None,
17    ),
18 )
```

دو نسخه مورد استفاده در آزمایش‌های این هفته در جدول زیر آمده‌اند:

نسخه	فایل	اندازه
1.1.0	esp32_ota_client-1.1.0.bin	892336 بایت
1.2.0	esp32_ota_client-1.2.0.bin	905952 بایت

Upload a new version
The SHA-256 is computed on the server after the file is stored.

Version
1.2.0

Changelog
OTA test release 1.2.0

Binary

Choose File

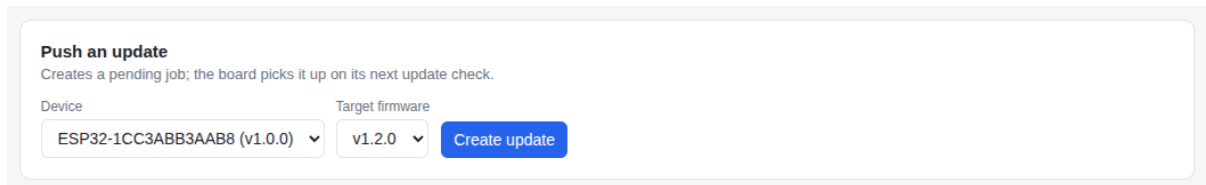
 esp32_ota_client-1.2.0.bin

Upload

شکل ۵: بارگذاری نسخه جدید ثابت‌افزار از طریق صفحه مدیریت نسخه‌ها در پنل

۳-۱. ایجاد عملیات بهروزرسانی دستی

برای هر آزمایش، دستگاه و نسخه مقصد از طریق پنل انتخاب شده و یک عملیات بهروزرسانی با حالت **manual** ایجاد شد. این عملیات در جدول **update_jobs** با وضعیت اولیه **pending** ثبت می‌شود و مورد در درخواست بعدی بررسی نسخه، اطلاعات همین عملیات را دریافت می‌کند.



شکل ۶: ایجاد عملیات بهروزرسانی دستی برای دستگاه و انتخاب نسخه ثابت‌افزار مقصد

۴. دریافت و نصب واقعی ثابت‌افزار روی مورد

پس از دریافت اطلاعات نسخه جدید، تابع **ota_install_from_url** فایل ثابت‌افزار را مستقیماً از سرور دریافت می‌کند. ابتدا پارتیشن مقصد با **esp_ota_get_next_update_partition** تعیین می‌شود و اندازه فایل با ظرفیت پارتیشن مقایسه می‌گردد. سپس عملیات نوشتن با **esp_ota_begin** آغاز شده و داده‌های دریافتی در بلوک‌های 4096 بایتی روی پارتیشن مقصد نوشته می‌شوند.

بخش اصلی حلقه دریافت و نوشتن به صورت زیر است:

```
1 while (1)
2 {
3     int read_len = esp_http_client_read(
4         client,
5         (char *)buffer,
6         OTA_BUFFER_SIZE
7     );
8
9     if (read_len < 0)
10    {
11        esp_ota_abort(ota_handle);
12        return ESP_FAIL;
13    }
14
15    if (read_len == 0)
16    {
17        break;
18    }
19
20    err = esp_ota_write(
21        ota_handle,
22        buffer,
23        read_len
24    );
25
```

```

26     if (err != ESP_OK)
27     {
28         esp_ota_abort(ota_handle);
29         return err;
30     }
31
32     total_written += read_len;
33 }

```

پس از اتمام دریافت، عملیات با تابع `esp_ota_end` نهایی می‌شود. سپس پارتیشن جدید با تابع `esp_ota_set_boot_partition` به عنوان پارتیشن بوت بعدی انتخاب می‌گردد:

```

1  err = esp_ota_end(ota_handle);
2  if (err != ESP_OK)
3  {
4      return err;
5  }
6
7  err = esp_ota_set_boot_partition(
8      update_partition
9  );
10 if (err != ESP_OK)
11 {
12     return err;
13 }

```

در این مرحله، نسخه قبلی حذف نمی‌شود و نسخه جدید در پارتیشن دیگر قرار می‌گیرد. این ساختار پایه لازم برای پیاده‌سازی Rollback در مراحل بعد پروژه را نیز فراهم می‌کند.

```

I (6773) main: Update available!
I (6773) main: Target version: 1.1.0
I (6773) main: Firmware ID: 1
I (6783) main: Firmware size: 892336 bytes
I (6783) main: Firmware URL: http://192.168.45.216:8000/api/firmwares/1/download
I (6793) main: SHA-256: 7d354ed0b2cc6df060ed9af779c3ee9428a33789d611d292b5aa749d8efa6fb7
I (6803) api_client: Reporting OTA status: downloading
I (6853) api_client: Response status: 200
I (6853) api_client: Response body: {"update_id":3,"status":"downloading"}
I (6863) main: Starting OTA update to version 1.1.0...
I (6863) ota_manager: Starting OTA update
I (6863) ota_manager: Firmware URL: http://192.168.45.216:8000/api/firmwares/1/download
I (6873) ota_manager: Target partition: ota_0
I (6913) ota_manager: Firmware size: 892336 bytes
I (9403) ota_manager: Written 4096 bytes
I (9573) ota_manager: Written 8192 bytes
I (9593) ota_manager: Written 12288 bytes
I (9613) ota_manager: Written 16384 bytes
I (9633) ota_manager: Written 20480 bytes
I (9663) ota_manager: Written 24576 bytes
I (9683) ota_manager: Written 28672 bytes
I (9713) ota_manager: Written 32768 bytes
I (9733) ota_manager: Written 36864 bytes
I (9763) ota_manager: Written 40960 bytes
I (9793) ota_manager: Written 45056 bytes
I (9823) ota_manager: Written 49152 bytes
I (9843) ota_manager: Written 53248 bytes
I (9863) ota_manager: Written 57344 bytes
I (9903) ota_manager: Written 61440 bytes

```

شکل ۷: دریافت فایل ثابت‌افزار از سرور و نوشتن مرحله‌ای آن روی پارتیشن مقصد

۵. نگهداری وضعیت عملیات و ثبت نتیجه پس از راه‌اندازی مجدد

یکی از نکات مهم در فرآیند به‌روزرسانی این است که وضعیت **SUCCESS** نباید پیش از راه‌اندازی واقعی نسخه جدید ثبت شود. برای این منظور، شناسه عملیات و نسخه مقصد پیش از راه‌اندازی مجدد در حافظه **NVS** ذخیره می‌شوند.

```
1 esp_err_t ota_state_save(  
2     int update_id,  
3     const char *target_version)  
4 {  
5     nvs_handle_t handle;  
6  
7     ESP_ERROR_CHECK(  
8         nvs_open(  
9             "ota_state",  
10            NVS_READWRITE,  
11            &handle  
12        )  
13    );  
14  
15    nvs_set_i32(handle, "update_id", update_id);  
16    nvs_set_str(handle, "target_ver", target_version);  
17    nvs_commit(handle);  
18    nvs_close(handle);  
19  
20    return ESP_OK;  
21 }
```

پس از راه‌اندازی مجدد، برنامه وضعیت ذخیره‌شده را می‌خواند. اگر نسخه در حال اجرا با نسخه مقصد برابر باشد، اجرای موفق نسخه جدید تأیید شده و وضعیت **SUCCESS** برای سرور ارسال می‌شود. پس از دریافت پاسخ موفق از سرور، وضعیت موقت از **NVS** پاک می‌شود.

```
1 if (ota_state.pending &&  
2     strcmp(  
3         firmware_version,  
4         ota_state.target_version  
5     ) == 0)  
6 {  
7     esp_err_t err = api_report_update_status(  
8         ota_state.update_id,  
9         "success",  
10        "New firmware booted successfully"  
11    );  
12  
13    if (err == ESP_OK)  
14    {  
15        ESP_ERROR_CHECK(ota_state_clear());  
16        ota_just_completed = true;  
17    }  
18 }
```


در طول فرآیند نیز وضعیت‌های **installing** و **downloading** برای سرور ارسال می‌شوند. در صورت بروز خطا هنگام دریافت یا نوشتن فایل، وضعیت **failed** گزارش می‌شود.

۵-۱. ثبت رویدادها در سرور

سرور هر تغییر وضعیت را در جدول **update_events** ثبت کرده و وضعیت جاری عملیات را در جدول **update_jobs** به‌روزرسانی می‌کند. در صورت دریافت وضعیت **SUCCESS**، نسخه فعلی دستگاه نیز به نسخه مقصد تغییر می‌کند.

```
1 conn.execute(  
2     "INSERT INTO update_events (update_id, status, message) VALUES (?, ?, ?)",  
3     (update_id, body.status, body.message),  
4 )  
5  
6 new_version = None  
7 if body.status == "success":  
8     new_version = job["target_version"]  
9  
10 conn.execute(  
11     """UPDATE devices  
12         SET last_update_status = ?,  
13             current_version = COALESCE(?, current_version),  
14             last_seen_at = CURRENT_TIMESTAMP,  
15             updated_at = CURRENT_TIMESTAMP  
16         WHERE id = ?""",  
17     (body.status, new_version, job["device_id"]),  
18 )
```

1	● ESP32-1CC3ABB3AAB8	1.0.0	manual ▾	pending	8/23/2026, 7:46:28 AM
1	● ESP32-1CC3ABB3AAB8	1.0.0	manual ▾	downloading	8/23/2026, 7:46:19 AM
1	● ESP32-1CC3ABB3AAB8	1.0.0	manual ▾	installing	8/23/2026, 7:46:28 AM

شکل ۸: نمایش تاریخچه مراحل عملیات به‌روزرسانی شامل **installing**، **downloading**، و **pending**

۶. جمع‌بندی هفته دوم

خروجی‌های اصلی هفته دوم عبارت‌اند از:

- بررسی ظرفیت واقعی حافظه Flash و اصلاح تنظیم پروژه از 2MB به 4MB؛
- طراحی و اعمال جدول پارتیشن شامل factory،ota_0،ota_1 و otadata؛
- تشخیص پارتیشن در حال اجرا و پارتیشن مقصد به‌روزرسانی؛
- تکمیل پردازش پاسخ JSON سرور و استخراج اطلاعات نسخه جدید با استفاده از cJSON؛
- ساخت و بارگذاری نسخه‌های جدید ثابت‌افزار روی سرور؛
- ایجاد و آزمایش عملیات به‌روزرسانی دستی از طریق پنل مدیریتی؛
- دریافت فایل ثابت‌افزار از سرور و نوشتن آن روی پارتیشن OTA؛
- انتخاب پارتیشن بوت جدید و راه‌اندازی موفق نسخه دریافت‌شده؛
- نگهداری وضعیت عملیات در NVS تا زمان تأیید راه‌اندازی نسخه جدید؛
- ثبت وضعیت‌های installing، downloading و success در سرور و تاریخچه عملیات؛
- آزمایش کامل مسیر به‌روزرسانی تا نسخه 1.2.0 و تأیید نتیجه در بورد، سرور، پایگاه داده و پنل.

در این مرحله مقدار SHA-256 فایل روی سرور محاسبه، ذخیره و همراه اطلاعات نسخه برای بورد ارسال می‌شود، اما مقایسه و تأیید این مقدار در سمت بورد هنوز پیاده‌سازی نشده است. همچنین مکانیزم Rollback و آزمایش کامل حالت به‌روزرسانی automatic به مراحل بعد پروژه منتقل شده‌اند. بنابراین خروجی هفته دوم، یک مسیر عملی و قابل آزمایش برای دریافت و نصب واقعی ثابت‌افزار و ثبت نتیجه عملیات است که زیرساخت لازم برای اضافه کردن بررسی صحت فایل و بازگشت به نسخه قبلی را فراهم می‌کند.